DSA CT-159

NED CAMPUS NAVIGATION SYSTEM FOR VISUALLY IMPAIRED STUDENTS

SUBMITTED TO:
DR. MUHAMMAD KAMRAN

HIBA ABRAR CT-204
EESHAH RASHID CT-213
FATIMA ZAMAN CT-205

TABLE OF CONTENTS

ABSTRACT

Problem Context

Navigating a large and complex campus like NED University is difficult for new students, visitors, and especially for visually impaired students. Public tools such as Google Maps are designed mainly for city-level roads and vehicle routes, not for internal campus walkways. They are also primarily visual and touch-based, which makes them inconvenient or inaccessible for blind users who rely on audio-based interaction.

Proposed Solution

This project presents a **graph-based, voice-guided campus navigation system** specifically tailored to NED University, with a strong focus on visually impaired users. The campus is modelled as a **weighted graph** built from a GeoJSON map of the area, where nodes represent locations and edges represent walkable paths.

Core **Data Structures and Algorithms (DSA)** concepts are applied:

- Graph representation using **adjacency lists**
- **Dijkstra's algorithm** for shortest path
- **Haversine formula** for geographic distance
- **Priority queues**, **hash maps** and **maps** for efficient lookup

The first version is a **C++ backend with step-based, voice-like console navigation**. The system loads the campus map, builds the graph, and provides text-to-speech-style instructions like "Turn left and walk 30 steps towards CSIT Labs."

A second version extends this logic into a **GPS-enabled, browser-based navigation interface** using HTML, JavaScript,

Leaflet.js, the Geolocation API, and Web Speech API. This version follows the user in real time using GPS and speaks turn-by-turn directions similar to Google Maps walking mode but restricted to NED's internal paths.

User Feedback from a Blind Student

An early step-based prototype was tested with a visually impaired student from NED. Her feedback was:

1. Interaction should feel like a **voice assistant** (Alexa-style), not a button-driven interface.
2. **Step counting is tedious** in real life; she preferred **GPS-based tracking** where the system automatically detects movement.

Outcome and Contributions

In response, the system evolved into a **fully voice-driven, GPS-aware campus navigation assistant**:

- **Version 1 – Step-Based C++ Prototype:**
  Demonstrates DSA concepts: graph modelling, Dijkstra, haversine distance, adjacency lists, etc.
- **Version 2 – GPS + Voice Navigation (Web + JS):**
  Uses real-time GPS position, voice input ("Start Navigation", "Use my GPS location"), and spoken directions suitable for visually impaired users.

Together, these versions show both the **theoretical DSA side** and the **practical accessibility-oriented application**.

1. PROBLEM UNDERSTANDING

(Criterion 1: Problem Understanding)

1.1 Background and Motivation

NED University has many departments, labs, hostels, sports areas, and internal roads. For sighted students, signs and visual maps are available, and they can also ask others for help. Visually impaired students, however, struggle to:

- Identify the correct building or gate
- Follow internal walkways without getting lost
- Navigate independently without human assistance

Traditional apps like Google Maps:

- Do not include **fine-grained internal walkways** of the NED campus
- Are optimized for **drivers and sighted users**
- Depend heavily on **visual screens and touch gestures**

This leads to reduced independence and confidence for blind students.

1.2 Navigation Challenges at NED University

Key challenges:

- Locating specific buildings (e.g., CSIT Labs, Library, Admin Block) for the first time
- Absence of a **detailed, accessible campus map** with internal pedestrian paths
- Lack of **voice-only, campus-aware navigation**
- No easy way to dynamically re-route if paths are blocked or the user deviates

1.3 Target Users and Accessibility Focus

- **Primary users:** Visually impaired students and visitors at NED University

- **Secondary users:** Any student wanting campus-specific walking navigation

Accessibility focus:

- Voice-first interface with minimal screen dependence
- Simple spoken instructions ("Turn left", "Continue straight for 40 steps")
- Support for real-time movement using GPS (in the advanced version)

1.4 Project Aims

The project aims to:

- Apply **DSA concepts** (graphs, shortest paths) to a **real, complex navigation problem**
- Provide **turn-by-turn, voice-style guidance** suitable for blind users
- Integrate **GPS** to track movement and adjust instructions
- Show how **user feedback** from a visually impaired student can reshape system design

---

2. REQUIREMENT IDENTIFICATION

(Criterion 2: Requirement Identification)

2.1 Formal Problem Statement

Design and implement a campus navigation system for NED University that:

1. Reads campus map data from a **GeoJSON** file.
2. Models campus locations and walkable paths as a **weighted graph**.
3. Computes **shortest walking routes** between any two campus locations.
4. Provides **step-by-step spoken or spoken-style instructions**.
5. Supports **voice-based interaction** and **GPS tracking** (in the advanced version).

2.2 Functional Requirements

- Load and parse **NED campus GeoJSON** data.
- Build a graph of **nodes (locations)** and **edges (paths)**.
- For the backend:
  - Allow users to list available locations.
  - Allow users to select **start** and **destination** by name.
  - Compute and output shortest path and distance.
  - Generate **turn-by-turn instructions** using bearings and distances.
- For the web + GPS version:
  - Provide HTTP APIs like /locations and /route (or equivalent).
  - Display routes on a **Leaflet map**.
  - Use **browser GPS** to track movement.
  - Use **Web Speech API** for voice input and output.
  - Recalculate route if user deviates significantly.

2.3 Non-Functional Requirements

- **Performance:** Dijkstra's algorithm should compute routes quickly enough for interactive use.
- **Correctness:** Routes must follow **actual walkable paths** from the GeoJSON, not straight lines through forbidden areas.
- **Reliability:** Handle missing inputs, invalid location names, and disconnected parts of the graph gracefully.
- **Usability:** Navigation should be **voice-first** and not demand precise screen touches.
- **Portability:** Frontend should run on standard mobile browsers.
- **Scalability:** New buildings and paths can be added by updating the GeoJSON.
- **Accessibility:** Designed with real feedback from a visually impaired student.

2.4 Assumptions and Limitations

• GPS accuracy is assumed to be sufficient outdoors on campus.
• Indoor navigation (inside buildings, floors, rooms) is out of scope. The current system only provides a simple static verbal hint for CSIT-related floors (e.g., how to reach the staircase), but it does not compute detailed indoor routes.
• Navigation is limited to the geographical bounding box of NED in the GeoJSON data.

---

3. SOLUTION OVERVIEW

(Criterion 3: Clarity of Solution)

3.1 High-Level Idea

1. **Convert campus map → graph:**
    - Extract walkable paths from export.geojson
    - Represent them as nodes (lat, lon, name) and edges (distance, open/closed)
2. **Compute route with Dijkstra:**
    - Given start and destination nodes, find minimum total distance path
3. **Generate human instructions:**
    - Convert geographic edges to **bearings** and **relative directions**
    - Convert distance to **steps** or meters
    - Speak instructions step-by-step
4. **Connect to frontend (advanced version):**
    - Backend returns route as list of coordinates
    - Frontend shows path on map, uses GPS and voice to guide user

3.2 Architecture

- **Backend (C++):**

    - Loads GeoJSON, builds graph, and runs Dijkstra.
    - Contains console navigation logic with “press space to continue” style interaction.
    - In the extended version, backend exposes HTTP endpoints for routes and locations.
- **Frontend (Web + JS, advanced):**
    - Map rendering with Leaflet.js
    - Voice input/output with Web Speech API
    - GPS tracking with Geolocation API
    - Calls backend APIs to obtain routes and location lists

3.3 How the Solution Addresses the Problem

- **Campus-specific:** Uses **real NED GeoJSON** instead of generic global maps.
- **DSA-driven:** Uses **graph algorithms** to always choose the shortest valid path.
- **Accessibility:** Uses **spoken guidance**, step lengths, and turn directions that are understandable without looking at the screen.
- **Extensible:** Can easily add indoor maps, multiple languages, or obstacle detection later.

---

4. DATA MODELLING AND CAMPUS GRAPH CONSTRUCTION

4.1 GeoJSON Data

- Input file: export.geojson containing LineStrings and building features around NED.
- Only features within a **bounding box** are considered to filter out unrelated city roads.

4.2 Node Representation

Each node is represented as:

struct Node {

```
    std::string name;
    double lat, lon;
};
```

- name: Human-readable label (e.g., “CSIT Labs” or “Path_Node_42”)
- lat, lon: Latitude and longitude coordinates

Stored in:

std::unordered_map<int, Node> nodes;

Key = integer node ID (0, 1, 2, …), value = Node.

4.3 Edge Representation and Graph Structure

Edges are represented as:

```
struct Edge {
    int to;
    double weight;
    bool open;
};
```

- to: Destination node ID
- weight: Distance (in meters) between nodes, computed using **haversine**
- open: If false, that path is considered closed

Graph representation:

std::vector<std::vector<Edge>> graph;

- graph[u] = list of all outgoing edges from node u
- This is an **adjacency list** representation.

4.4 Constructing the Graph from LineStrings

For each **LineString** in the GeoJSON:

1. Extract coordinates inside NED’s bounding box.
2. For each coordinate:
    - Check if a node already exists within ~2 meters (deduplication)
    - If not, create a new node and assign a Path_Node_x name

3. Connect consecutive coordinates as edges in **both directions** with haversine distance as weight (if reasonable, e.g., < 500 m).

4.5 Landmark Naming with Predefined Locations

A global map of important locations is defined:
std::map<std::pair<double,double>, std::string> predefined_locations;
For each predefined (lat, lon, name):

- Find nearest existing node within a small radius (e.g., < 15–20 m).
- Rename that node to the building name (e.g., "NED University Library").
- If no nearby node exists, create a new node for that building and connect it to closest path node.

This ensures that:

- Real-world building names show up in menus
- Navigation instructions refer to meaningful locations instead of raw coordinates

4.6 Fixing Isolated Locations

Some nodes may be isolated due to data gaps. The function:
void fix_isolated_locations(unordered_map<int, Node>& nodes,
vector<vector<Edge>>& graph)
adds manual connections (predefined node ID pairs) and creates edges using haversine distance. This ensures the graph is **fully connected** for expected paths.

5. DSA CONCEPTS AND DESIGN DECISIONS

5.1 Core Data Structures

1. **Adjacency List Graph – vector<vector<Edge>>**
    - Efficient for sparse graphs (campus network).
    - Easy to iterate over all neighbours of a node.
    - Memory usage proportional to actual edges (O(V + E)).
2. **Node Table – unordered_map<int, Node>**
    - Fast O(1) average lookup by ID.
    - Allows flexible node IDs that are not necessarily dense or ordered.
3. **Predefined Locations – map<pair<double,double>, string>**
    - Stores known named buildings by coordinate.
    - map gives deterministic order for menus and stable iteration.
4. **Coordinate Deduplication – map<pair<double,double>, int>**
    - Used to map coordinate pairs to node IDs while building the graph.
    - Prevents creation of multiple nearly-identical nodes for the same physical point.
5. **Priority Queue for Dijkstra – priority_queue<pair<double,int>, ...>**
    - Always selects node with smallest current distance.
    - Time complexity O(E log V).

6. **Parent and Distance Arrays – vector<int> parent, vector<double> dist**
    - Store shortest distances and predecessor nodes for path reconstruction.
7. **Miscellaneous Vectors and Maps**
    - vector<int>: for storing paths and search results.
    - map<int, string>: for menu options in console UI.

5.2 Key Algorithms

1. **Haversine Distance**
    - Given two (lat, lon) pairs, compute distance in meters.
    - Used as **edge weight** in the graph.
2. **Dijkstra's Shortest Path Algorithm**

```
vector<double> dijkstra(int n,
                        const vector<vector<Edge>>& graph,
                        int src,
                        vector<int>& parent);
```

- Initialises all distances to INF, except src = 0.
- Uses a **min-heap (priority queue)** for picking the next closest node.
- Updates dist[v] and parent[v] whenever a shorter path is found.
- Skips edges where e.open == false.

3. **Path Reconstruction**
    - Starting from destination t, follow parent[t] back to src.
    - Reverse the resulting list to get route from start to destination.
4. **Bearing Calculation and Turn Detection**
    - For nodes (lat1, lon1) → (lat2, lon2) compute bearing in degrees (0–360).
    - Compare consecutive bearings to decide whether the instruction is:

        - Continue straight
        - Turn left
        - Turn right
    5. **Distance to Steps Conversion**
        - Approximate step length (0.762 m) converts meters to "N steps".
        - Used in console prototype to give user **step-wise instructions**.

5.3 Why These DSA Choices and Not Others

- **Adjacency List vs Adjacency Matrix**
    - Campus graph is sparse; adjacency matrix would waste memory ($O(V^2)$).
    - Adjacency list stores only existing edges, enabling faster traversal.
- **Dijkstra vs BFS vs Floyd–Warshall**
    - BFS assumes unweighted edges; campus paths have different distances.
    - Floyd–Warshall computes all-pairs shortest paths with $O(V^3)$ complexity, overkill for on-demand routing.
    - Dijkstra + priority queue is optimal here: **O(E log V)** and easy to implement.
- **unordered_map vs map for nodes**
    - Node IDs are looked up frequently; O(1) average time is important.
    - Sorting by ID is not a requirement, so we prefer hash table.
- **Priority Queue vs Simple Queue / Stack**
    - Correctness of Dijkstra depends on always extracting the smallest distance node next, which a simple queue or stack cannot guarantee.

These choices directly support good **time and space complexity** for a complex computing problem (graph routing on real map data).

---

6. ROUTING AND DIRECTION GENERATION

6.1 Running Dijkstra

Given:

- start_id (source node)
- dest_id (destination node)

Call:

```
vector<int> parent;
vector<double> dist = dijkstra(graph.size(), graph, start_id, parent);
```

- If dist[dest_id] is INF, there is no path.
- Otherwise, reconstruct path using parent.

6.2 Path Reconstruction

```
vector<int> path;
for (int cur = dest_id; cur != -1; cur = parent[cur]) {
    path.push_back(cur);
}
reverse(path.begin(), path.end());
```

This gives a sequence of node IDs representing the shortest route.

6.3 Bearing and Turn Instructions

For each consecutive pair (u, v) in the path:

1. Compute distance w using find_edge_weight(graph, u, v).

2. Compute bearing between (nodes[u].lat, nodes[u].lon) and (nodes[v].lat, nodes[v].lon).
3. Compare with previous bearing to decide **turn left/right/straight**.
4. Convert distance w into **steps** using format_steps(w).

Example spoken instruction:
"Start walking for 35 steps towards CSIT Labs."
"Turn right and continue for 25 steps towards pathway."
Path_Node_x names are hidden in instructions and replaced with generic "pathway" for clarity.

---

7. BACKEND IMPLEMENTATION – STEP-BASED C++ CONSOLE VERSION

7.1 Main Components

The step-based C++ console backend is organised into the following main components:

- **load_map()**
  – Reads the export.geojson file.
  – Filters features to the NED campus bounding box.
  – Builds the graph of nodes (locations) and edges (walkable paths).
  – Assigns human-readable names to nodes when they are close to predefined landmarks (e.g., "NED University Admin Block", "CSIT Labs").

- **fix_isolated_locations()**
  – Connects a small manually curated list of node ID pairs that are known to be connected on campus.

– Uses haversine distance to create bidirectional edges between these nodes.
– Ensures the internal campus graph does not contain isolated components along important routes.

• **speak(), wait_for_space(), get_numeric_input()**
– **speak():** Simulates voice output by printing “SYSTEM: …” with a short delay to mimic text-to-speech.
– **wait_for_space():** Waits until the user presses SPACE and ENTER, used to control step-by-step interaction.
– **get_numeric_input():** Safely reads numeric choices from the user (e.g., main menu options), re-prompting on invalid input.

• **list_locations_menu()**
– Iterates through the predefined list of campus locations (Admin Block, Library, CSIT Labs, DMS Cafeteria, etc.).
– Prints only those that actually exist in the built graph.
– Paginates the list, asking the user to press SPACE to continue after a few entries.
– Uses speak() to “announce” sections of the list in a voice-style manner.

• **get_location_from_user()**
– Prompts the user to type a location name (e.g., “CSIT Labs”, “NED University Library”).
– First checks for an exact match; if none, searches for partial matches.
– If one partial match is found, it confirms and uses that location.
– If multiple matches are found, it reads them out and asks the user to be more specific.

– For CSIT-related locations (“CSIT Labs”, “CSIT Department Entrance”, “CSIT Offices”, etc.), it additionally asks the user which floor they are on (first/second/third) and gives a simple static hint about how to reach the staircase. This is a verbal aid only, not full indoor routing.

• **navigation_mode()**
– Uses **get_location_from_user()** to obtain start and destination locations by name.
– Looks up the corresponding node IDs in the graph.
– Runs Dijkstra’s algorithm from the start node to compute shortest distances to all nodes.
– Checks if a valid path to the destination exists.
– Reconstructs the shortest path using the parent[] array.
– Announces the total route distance in human-readable units.
– Generates step-by-step guidance:
▫ Computes bearings between consecutive nodes to decide whether to say “Start walking”, “Continue straight”, “Turn left”, or “Turn right”.
▫ Converts the edge distance into approximate steps using average step length.
▫ Hides internal Path_Node_x names and instead refers to them as “pathway” in instructions.
▫ For each step, allows the user to press N for the next instruction or R to repeat the current one.
– Ends with “You have arrived at your destination.”

• **set_edge_open()** and **check_edge_status()**
– set_edge_open(): Marks edges between two node IDs as open or closed in both directions.
– check_edge_status(): Reports whether an edge exists and whether it is currently open or closed.

– These functions are used by the “Path tools” menu to simulate path closures or re-openings (e.g., construction blocks a route).

• **main()** and Main Menu
– Loads the map and fixes isolated locations.
– Starts the console interaction loop and presents the main menu:
  ▫ 1: List locations
  ▫ 2: Navigate
  ▫ 3: Path tools (open/close/check edges)
  ▫ 4: Reload map
  ▫ 5: Exit
– Routes each choice to the appropriate functions described above.

---

8. GPS-ENABLED AND VOICE-DRIVEN EXTENSION (WEB VERSION)

8.1 Additional Backend Role

In the GPS-based version, the backend plays two roles:

1) Route Computation and Location Data
• Provides logical routes between named locations on campus.
• Uses the same graph and Dijkstra-based routing logic as the console version.
• (In an HTTP-based setup) can be wrapped behind simple APIs such as /locations and /route to serve:
– A list of named locations (e.g., Admin Block, Library, CSIT Labs, DMS Cafeteria).

– A shortest path between two locations as an ordered list of coordinates plus total distance.

2) GPS Helper Functions

• Maintains the last known GPS reading (latitude, longitude, and accuracy) from the user's device.
• Provides a helper to find the nearest predefined landmark to the current GPS position.
• Computes the distance from the current GPS position to a named destination on campus.
• Checks whether the user has "arrived" at the destination by comparing this distance against a small threshold (e.g., within 20 meters).
• Returns a simple text description of the current GPS state (latitude, longitude, accuracy) for debugging or status display.

These GPS helpers are designed to be used by the web frontend. The frontend can:
• Update the backend with the current GPS location.
• Ask how far the user is from the destination.
• Use the "arrival" check to decide when to announce that the user has reached their goal.

---

9. PROTOTYPE EVOLUTION AND USER STUDY

- **Version 1 – Step-Based Console:**
  - Fully demonstrates Dijkstra and graph-based routing.
  - Uses "walk N steps" type instructions.
  - Requires keyboard button presses to proceed.
- **User Study with Visually Impaired Student:**

    - She liked the concept of an NED-specific navigation tool.
    - Requested **voice-assistant-like interaction** and removal of manual step counting.
- **Version 2 – GPS + Voice:**
    - Uses real-time GPS instead of step counting.
    - Trigger phrase "Start Navigation."
    - Fully voice-driven; user need not look at or touch the screen frequently.

This evolution shows how **DSA-correct but user-unfriendly** designs must be refined to meet **human and accessibility requirements**.

---

10. TESTING AND USER FEEDBACK

10.1 Purpose of Testing

The testing phase aimed to evaluate the accuracy, usability, and accessibility of the Campus Navigation System, especially for visually impaired users. The goal was to ensure that the system provides safe, clear, and reliable navigation across the campus and that all core functions—pathfinding, voice guidance, and GPS integration—work as intended.

10.2 Testing Methodology

10.2.1 Functional Testing

Functional testing was conducted to verify individual components of the system. The following features were evaluated:

- Shortest path generation
- Handling of predefined and custom locations
- Voice-based navigation instructions
- Input validation for unknown or invalid locations
- GPS permission handling
- Recalculation for alternative paths

No external automated tools were used; the testing was performed manually on the integrated frontend-backend setup.

10.2.2 Usability Testing

Usability testing was carried out with two groups:

1. A visually impaired (blind) student who tested the system while fully relying on audio output.
2. Random sighted students who evaluated ease of use, clarity of interface, and accuracy.

The visually impaired testing session lasted approximately 15–20 minutes, conducted on campus under safe supervision.

10.3 Test Scenarios

Scenario 1 (Blind User Test)

- Route: CSIT Labs → DMS Cafeteria
- Purpose: Evaluate navigation clarity and accessibility for a visually impaired student.

Scenario 2

- Route: CSIT Department → Library
- Purpose: Test general usability and shortest-path accuracy.

Scenario 3

- Route: CSIT Department → Civil AV Hall
- Purpose: Verify voice directions for longer routes.

Scenario 4

- Invalid location input (e.g., “xyz hall”)
- Expected: Error message + guidance to valid inputs.

Scenario 5

- GPS disabled scenario
- Expected: App prompts user to enable location permissions.

10.4 Interface Walkthrough

Figure 1 – Voice Navigation Welcome Prompt and Command Hints

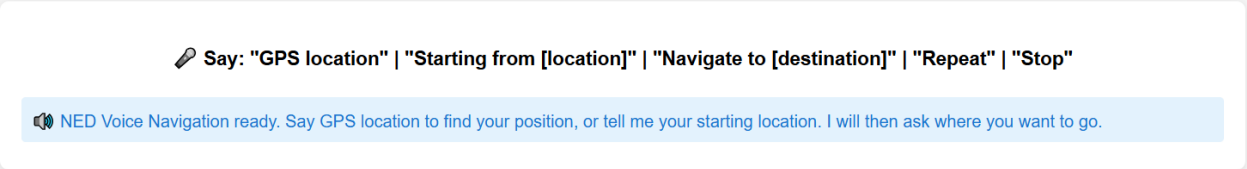

Figure 2 – Start Location Detected: “CSIT Labs” (Waiting for Floor Input)

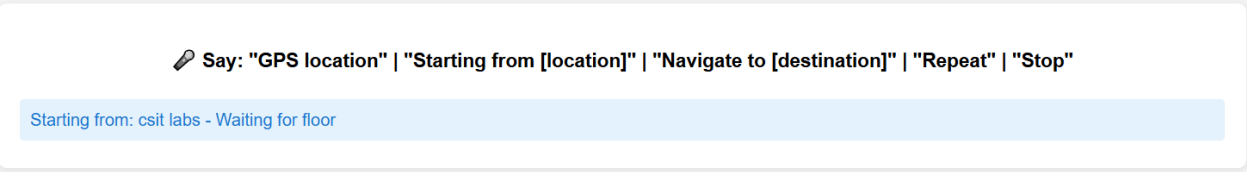

Figure 3 – Floor Selection Confirmed and Destination Prompt

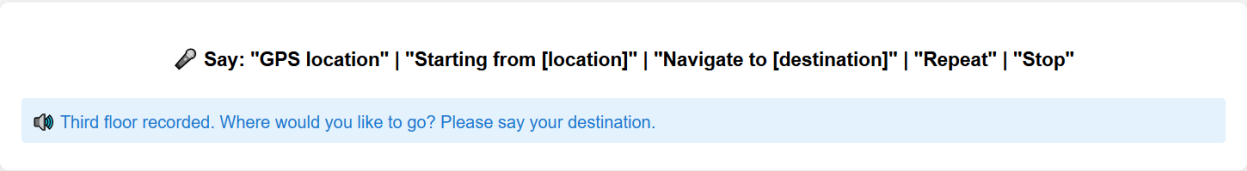

Figure 4 – Generated Turn-by-Turn Navigation Instructions (CSIT Labs to Civil AV Hall)

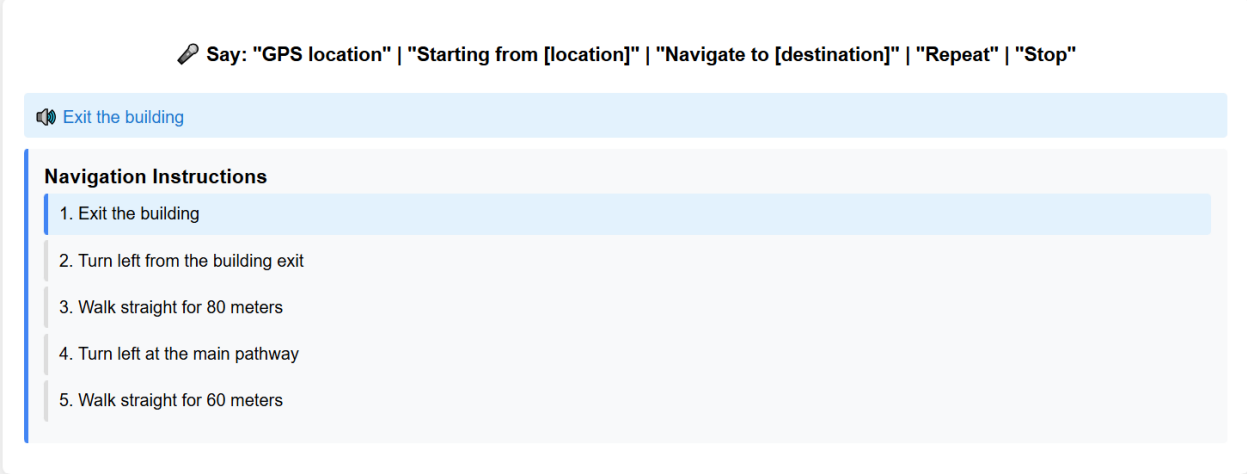

Figure 5 – NED University Voice-Controlled Map Interface with Route from CSIT Labs to Civil AV Hall

10.5 User Feedback

10.5.1 Blind Student Feedback

Route Tested:
CSIT Labs → Civil AV Hall

Shortest path calculation	Correct path generated	All paths matched expected shortest routes	Pass
Voice navigation	Clear and timely instructions	Instructions clear but	Pass

		slightly fast in long hallways	
Invalid input handling	Show error plus suggestions	Worked as expected	Pass
GPS permissions	Fetch location or prompt user	Prompt displayed successfully	Pass
Predefined locations	Accurate mapping	All predefined points detected correctly	Pass

Feedback Summary:

- The voice instructions were understandable and easy to follow.
- The user felt confident while moving, especially with the step-by-step guidance.
- The student suggested giving earlier warnings before turns (“say turn slightly sooner”).
- They appreciated the simplicity of start-to-finish instructions without overloading detail.
- They mentioned the app felt “helpful and comfortable” for campus use.

10.5.2 Sighted Students Feedback

Routes Tested:

- CSIT Department → Library
- CSIT Department → Civil AV Hall

Feedback Summary:

- Map paths matched real campus distances well.
- Instructions were smooth, but a few students suggested adding a visual map preview.
- Timing of voice prompts was considered good.
- Students found the interface simple and straightforward.
- Some recommended adding an option to repeat the last instruction.

10.6 Observations

- The visually impaired user relied heavily on voice timing → instructions may need slightly earlier cues.
- Predefined locations made testing easier and ensured consistency.
- GPS permission prompts worked, but could be improved with clearer guidance.
- The backend shortest-path logic performed consistently with no misdirection.
- Users preferred simple phrasing over complex directions.

10.7 Improvements Suggested

Based on testing and feedback:

1. Add vibration feedback for turn alerts (optional).
2. Add “Repeat last instruction” voice command.
3. Slow down voice instructions slightly in noisy environments.
4. Provide earlier turn notifications.
6. Improve phrasing for some instructions to make navigation more natural.

10.8 Conclusion

Testing confirmed that the system works accurately and is usable by both visually impaired and sighted individuals. The blind student was able to navigate independently with confidence, and all functional tests met their expected outcomes. Feedback from users helped highlight key areas for improvement—mainly timing adjustments and optional extra features—ensuring that the system moves closer to becoming a fully reliable assistive navigation tool for campus-wide use.

Blind Student Feedback (Questionnaire)

During the usability testing session with a visually impaired student, several insights were gathered regarding her navigation experience, comfort level, and comparison with existing tools.

Key Points Mentioned by the Student

1. Better Than Google Maps

- She shared that Google Maps often gives only broad locations like “Library” or “Cafeteria” without specific campus-level directions.
- Maps didn’t guide her inside campus pathways and lacked turn-by-turn instructions within buildings or smaller routes.
- She appreciated that our system provides highly detailed, campus-specific instructions, making it easier to understand where to go inside the university environment.

2. Felt Safe and Comfortable

- She mentioned that she felt safe throughout the navigation process.
- The guidance felt consistent and didn’t confuse her.
- She said the instructions were simple enough that she didn’t feel stressed or overwhelmed.

3. Clear and Helpful Voice Instructions

- Her favourite part was the voice instructions, especially the step-by-step clarity.
- She liked that the system “talked her through the route” rather than giving vague hints.
- She recommended slightly earlier cues before turns, but overall she found the voice output very clear.

4. Willingness to Use Regularly

- She stated that she would definitely want to use the application regularly on campus.
- She said this system felt more approachable and relatable to her daily movement needs than general navigation apps.

10. EVALUATION AND RESULTS

10.1 Correctness

- Tested routes between multiple pairs of locations (e.g., Main Gate → Admin Block, Admin Block → Library, Library → CSIT Labs).
- Routes follow campus walkways defined in GeoJSON, not straight lines through buildings.
- If a path is manually marked as closed, Dijkstra automatically finds alternative routes.

10.2 Performance

- Campus graph size (hundreds of nodes, thousands of edges) is well within Dijkstra's practical limits.
- Route computation happens in a fraction of a second on a normal laptop, suitable for interactive use.

10.3 User Experience

- **Console version:** Instructions are clear but step counting is not ideal in real life.
- **GPS + Web version:** Directions arrive at more natural times (just before turns), which matches user mental model of navigation apps.

11. DISCUSSION

11.1 Strengths

- Uses **real data (GeoJSON)** instead of toy graphs.
- Demonstrates clear use of **DSA concepts** in a complex computing problem.

- Integrates **voice, GPS, and pathfinding** for accessible navigation.
- Involves **actual user feedback** from a visually impaired student.

11.2 Limitations

- Dependent on GPS accuracy (may be worse near tall buildings).
- No detailed indoor navigation yet.
- Voice recognition quality depends on environmental noise and accents.

11.3 Lessons Learned

- "Correct" algorithms alone are not enough; **usability and accessibility** matter.
- Operating on **real-world data** reveals edge cases (isolated nodes, noisy coordinates).
- Iterative design with user feedback leads to much better solutions.

---

12. CONCLUSION

This project models NED University's campus as a **weighted graph** and applies **Dijkstra's shortest path algorithm** with **haversine distance** to compute realistic walking routes. It then transforms these routes into **turn-by-turn, voice-style instructions** for visually impaired users.

Starting from a step-based C++ console prototype, the work evolved into a **GPS- and voice-based navigation system** similar to Google Maps walking mode but specialized for NED's internal paths. The project successfully demonstrates how **core DSA topics** (graphs, shortest paths, priority queues, hashing) can be applied to solve a **real, accessibility-focused problem**.

---

13. FUTURE WORK

- **Indoor Navigation and Floor-Level Routing**
  Extend the current graph model to cover building interiors, including corridors, staircases, elevators and individual floors. Each floor can be represented as a separate sub-graph, with vertical connections (stairs/lifts) modelled as weighted edges. This would allow visually impaired users to get guidance not only to "CSIT Labs" but also to specific rooms or labs on a particular floor.
- **Dynamic Obstacle Detection and Safety Alerts**
  Integrate camera-based or external sensor inputs (e.g., smartphone camera, smart cane, or wearable sensors) to detect temporary obstacles such as parked bikes, construction work, or crowds. When an obstacle is detected along the calculated route, the system could alert the user and either suggest a safer detour or slow, careful movement with explicit warnings.
- **Language Support and Localisation (Urdu and Others)**
  Add multilingual support for both speech recognition and speech synthesis, starting with Urdu and possibly other regional languages. This includes localising location names, instructions, and system prompts so that users can interact in their preferred language while still keeping English as an option for international students and visitors.
- **Native Mobile Applications (Android / iOS)**
  Wrap the web-based interface into native Android and iOS applications. Native apps would provide better access to device sensors (GPS, camera, vibration), background operation for continuous navigation, and tighter integration with accessibility features such as TalkBack and VoiceOver.

- **Offline and Low-Connectivity Mode**
  Enable offline usage by caching the GeoJSON map, the campus graph, and frequently used routes on the device. In low-connectivity or no-internet situations, the system should still be able to compute routes locally and provide voice guidance, synchronising any logs or updates once a connection is restored.
- **Visual Campus Boundary Highlighting on the Map**
  Add a clearly visible red outline around the official NED campus boundary on the map. This boundary layer would help sighted users and instructors quickly distinguish "inside campus" from surrounding city roads, and it can also be used programmatically to ensure that routing is constrained to paths that lie within the campus area.

---

14. REFERENCES

- Leaflet.js documentation (map rendering)
- GeoJSON specification (geospatial data format)
- Web Speech API documentation (speech recognition and synthesis)
- Geolocation API documentation (GPS access)
- Standard descriptions of Dijkstra's algorithm and the haversine formula